

universidade de aveiro



deti

departamento de eletrónica,
telecomunicações e informática

Software Engineering

LLMs in Software Development Lifecycle

November 2025 / 2025/2026



Outline

- Introduction
- Using AI in SDLC
 - Areas
 - Use cases
 - Concerns using AI in SDLC
 - Agents in SDLC

Who use AI in SDLC? *[In the media..]*



Mark Zuckerberg wants AI to do half of Meta's coding by 2026

The comments came during a discussion between Zuckerberg and Microsoft CEO Satya Nadella.

By [Cecily Mauran](#) on April 29, 2025



Zuckerberg provided new insights on the future of AI coding at Meta. Credit: Chris Unger / Zuffa LLC / Getty Images

BENZINGA

Anthropic CEO Says AI Could Write '90% Of Code' In '3 To 6 Months'—Warns 'Every Industry' Will Be Affected

LaToya Scott

March 27, 2025 • 3 min read



GitHub CEO: Future devs will not code, they will manage AI

Gartner reports - 2025

Gartner Predicts 80% of Enterprise Software and Applications Will Be Multimodal by 2030, Up from Less Than 10% in 2024

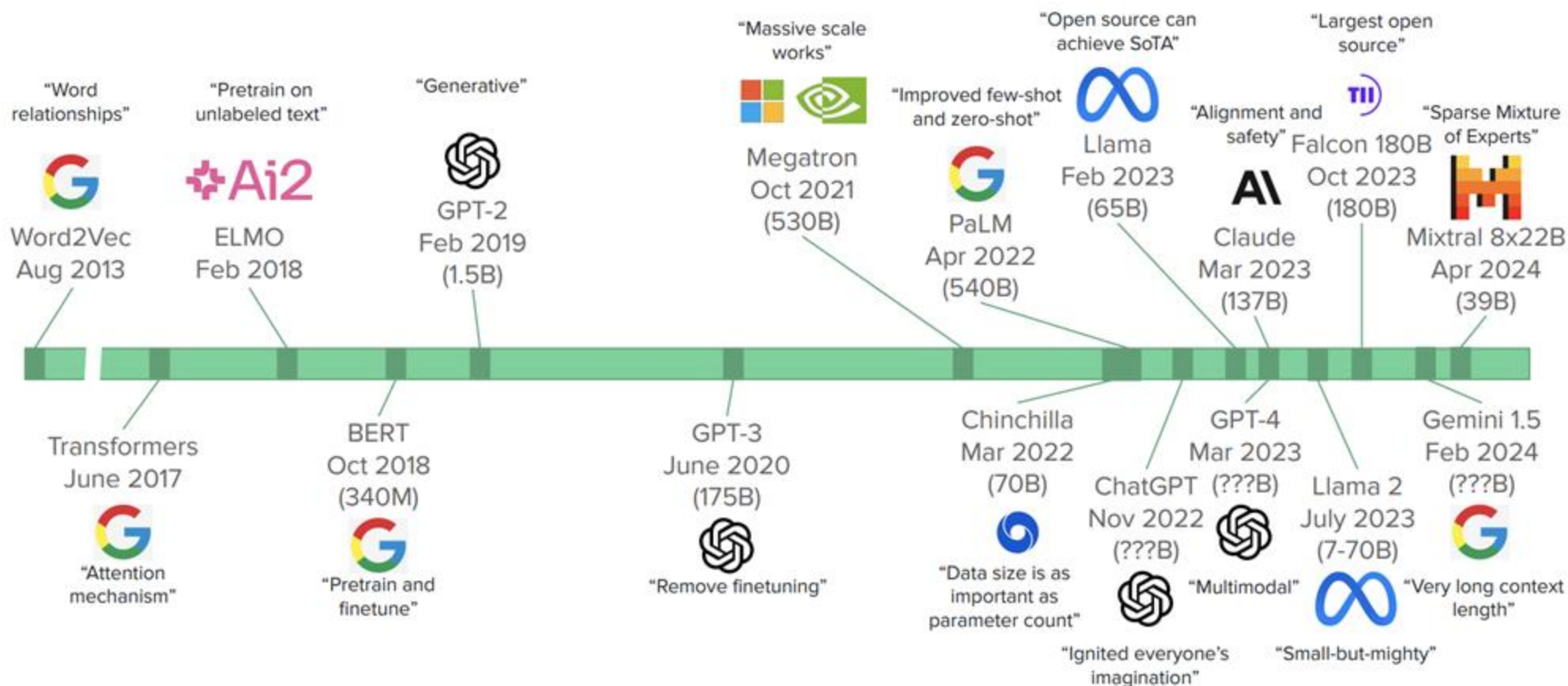
STAMFORD, Conn., July 2, 2025

Over the Next One to Three Years, Multimodal GenAI Models Will Increasingly Enrich More Applications

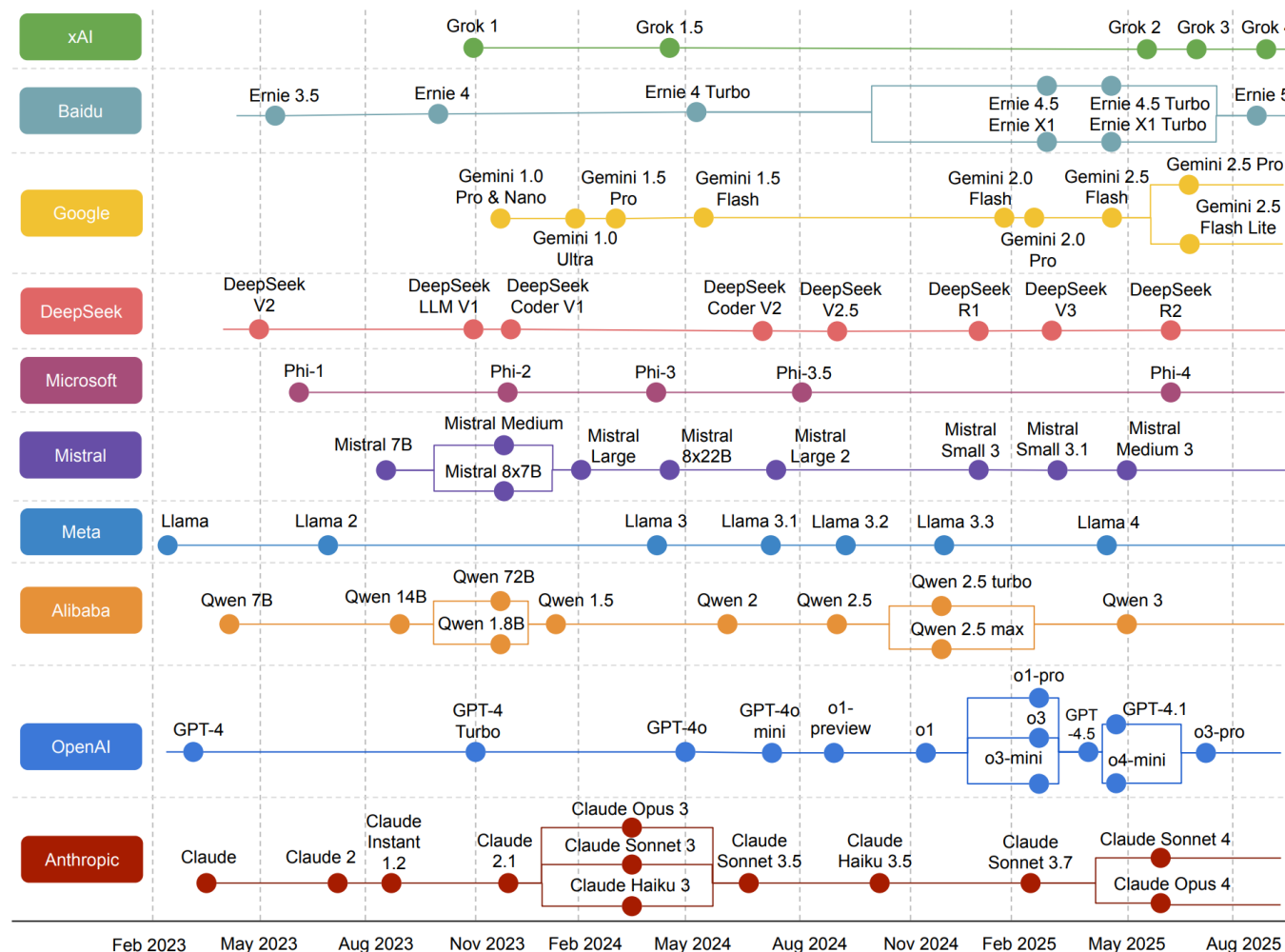
Eighty percent of enterprise software and applications will be multimodal by 2030, up from less than 10% in 2024, according to Gartner, Inc.

"The shift to multimodal enterprise software is a fundamental transformation in business operations and innovation," said [Roberta Cozza](#), Sr. Director Analyst at Gartner. "Multimodal generative AI (GenAI) will revolutionize enterprise applications by adding previously unattainable features and functionalities, impacting sectors like healthcare, finance, and manufacturing. By enhancing domain-specific language models, it will improve accuracy, automate operations, and drive contextual decision intelligence, enabling AI to take proactive actions across tasks."

LLMs evolution (1/2)



LLMs evolution (2/2)

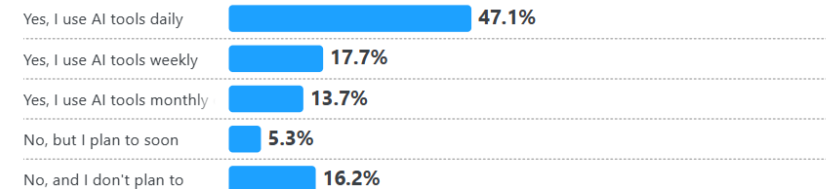


Who use AI in SDLC?

2025, Stack Overflow received over **49,000+ responses** from **177 countries** across 62 questions focused on 314 different technologies, including **new focus** on **AI agent tools, LLMs** and community platforms.



84% of respondents are using AI tools this year



84% of respondents are using or planning to use AI tools in their development process, an increase over last year (76%). This year we can see 51% of professional developers use AI tools daily.

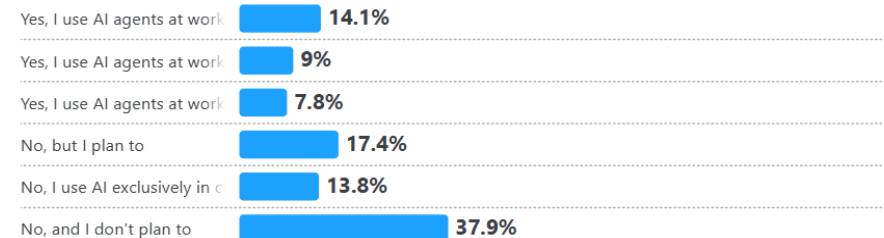
<https://survey.stackoverflow.co/2025>

Who use AI in SDLC?

2025, Stack Overflow received over **49,000+ responses** from **177 countries** across 62 questions focused on 314 different technologies, including **new focus** on **AI agent tools, LLMs** and community platforms.



A majority of developers don't use AI agents



AI agents are not yet mainstream. A majority of developers (52%) either don't use agents or stick to simpler AI tools, and a significant portion (38%) have no plans to adopt them.

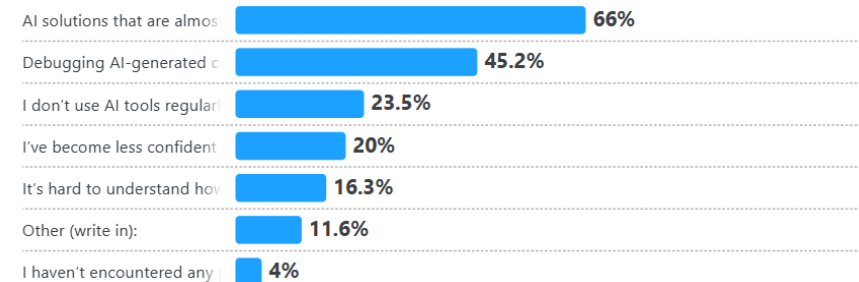
<https://survey.stackoverflow.co/2025>

Who use AI in SDLC?

2025, Stack Overflow received over **49,000+ responses** from **177 countries** across 62 questions focused on 314 different technologies, including **new focus** on **AI agent tools, LLMs** and community platforms.



66% of developers are frustrated with AI solutions that are almost right

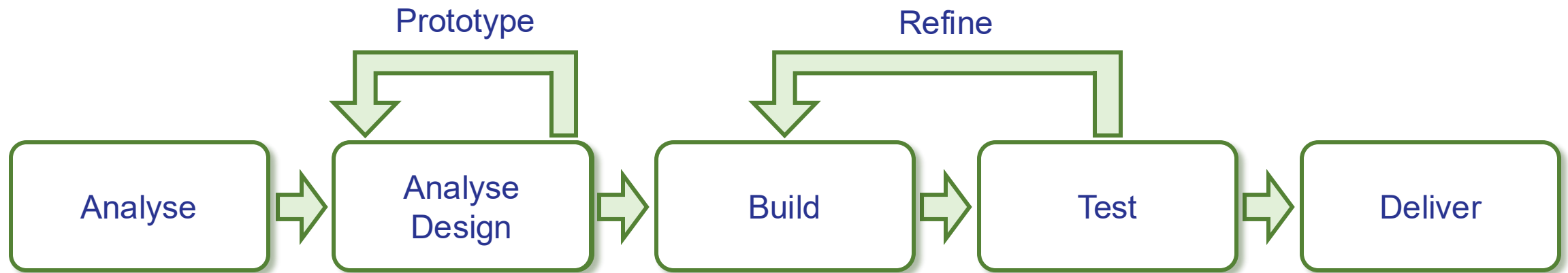


The biggest single frustration, cited by 66% of developers, is dealing with "AI solutions that are almost right, but not quite," which often leads to the second-biggest frustration: "Debugging AI-generated code is more time-consuming" (45%)

<https://survey.stackoverflow.co/2025>

SDLC - Iterative cycles

This area is undergoing rapid and continuous transformation.



In all the phases, AI with LLMs or Agentic can be used.

Areas that are possible to use AI in SDLC

- Definition of requirements and user stories *[to help on write]*
- Autocomplete code
- Code generation
- User Interface and Experience
- Bug Detection
- Code Review
- Automate Testing & Quality Assurance
- Predictive Analytics and Performance Optimization
- Documentation & Technical Writing
- Chatbot and Virtual Assistance (e.g. Agents, etc)

Example - LLMs to Summarize Meetings

1. **Our Notes:** Focus, critical thinking, preparation
2. **LLM Notes:** Efficiency, structured records, less effort
3. **Trade-off:** **Do we lose focus if LLMs take over?**



Best Mix: Quick personal notes + LLM full summary

Use Cases

➤ Generating code snippets

Pros: Speeds up prototyping and development, supports multiple languages, bridges idea-to-implementation gap

Cons: **Risk of incorrect or insecure code**, may reduce deep understanding of the logic

➤ Debugging

Pros: Saves time, **explains errors**, improves long-term debugging skills

Cons: **May miss subtle or context-specific bugs, can lead to over-reliance on AI suggestions**

➤ Accelerate learning

Pros: Fast onboarding to new tech, provides examples/tutorials, summarizes documentation effectively

Cons: **Learning can become passive**, advice **quality** depends on AI model's accuracy and knowledge base

➤ Optimizing code

Pros: Improves performance, readability, and maintainability; helps reduce technical debt early

Cons: Suggestions may not fit the project context, **risk of over-optimization or unnecessary changes**

➤ Automating documentation

Pros: Saves time, keeps docs consistent with code, improves team collaboration and knowledge transfer

Cons: May **produce generic or inaccurate descriptions**, still **requires human validation and review. Eg LLMS usually write (or seems to be very good writing text, but it can be distracted!)**

Available tools in the market

➤ Browser-Based Tools

- ChatGPT
- Google Gemini
- ...

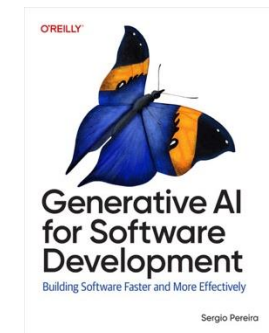
➤ IDE-Based Tools

- GitHub Copilot
- Claude Code
- Cursor
- Windsurf
- ...

How to evaluate this tools?

- Define criteria's
- Define problem (e.g. 2D Puzzle, ToDo)
- Define prompt / replicate across tools
- Measure and compare results

Tool	UX	Test performance
ChatGPT	Browser	8/10
Google Gemini	Browser	4/10
GitHub Copilot	IDE	8/10
Cursor	IDE	9/10
Windsurf	IDE	9/10



Tested April 2025.

LLM Code Tool - Comparison

Simple classic-style coding tasks:

- ▶ All **5 tested tools solved algorithmic puzzles** (clear I/O, single file, common patterns) in **~10 seconds**.
- ▶ **Outperformed human engineers** for these narrow, well-defined challenges.

Larger codebases & projects:

- ▶ Tools struggled with multi-file projects (e.g., Kanban UI + backend).
- ▶ Browser-based tools (ChatGPT, Gemini) make code integration cumbersome due to copy/paste workflow.

IDE vs. Browser experience:

- ▶ IDE-based tools (Cursor, Windsurf, GitHub Copilot): Seamless editing, review, and testing inside context.
- ▶ Browser-based tools: Closed model ecosystem, heavy context sharing burden, poor for large projects.

Tool differentiation:

- ▶ GitHub Copilot: Chat-first integration inside IDE.
- ▶ Windsurf: Agentic approach: creates folders/files, review & “accept all” workflow.
- ▶ Cursor: Balanced chat + optional agentic mode.

Structuring the prompt (1/2)

- Be clear & specific (define task, context, format)
- Use step-by-step instructions when complex
- Provide examples of input/output when possible
- Keep iterating & refining the prompt
- Tools like GitHub Copilot assist with context-aware prompts in code

Three “S” principle, *(from GitHub Copilot)*

SIMPLE | SPECIFIC | SHORT

Structuring the prompt (2/2) [Application]

Given the following error messages, logs, and user description, identify the most likely technical system issues, ordered by probability (from most to least likely).

For each identified issue, provide:

- *A clear description of the probable root cause*
- *A technical justification based on the provided information*
- *Relevant excerpts copied exactly from the original logs or text*
- *A category code (e.g., “network”, “storage”, “permissions”, “application”, “database”)*

Format your response as a JSON array where each issue contains:

- *description: text describing the probable cause*
 - *justification: technical reasoning*
 - *highlight: a list with the exact excerpts from the logs or user message that support the conclusion*
 - *category: a short category label*
-

Context Window (1/2)

The context window is the maximum amount of text (input + output) an LLM can process at once.

For example:

- ▶ GPT-4 Turbo: ~128k tokens (~300 pages of text)
- ▶ Claude 3.5 Sonnet: ~200k tokens
- ▶ Smaller models: often 4k–32k tokens
- ▶ Once the window limit is reached, older parts of the conversation fall out of memory unless explicitly re-provided.

Context Window (2/2)

SDLC Stage	Context Window Use	Risk / Limitation	Mitigation
Requirements Analysis	Load entire spec, user stories, regulatory docs for analysis.	Large projects may exceed token limit.	Chunk documents and summarize; use retrieval-augmented generation (RAG).
Design	Include architecture diagrams, APIs, design patterns for review or generation.	May lose prior context when switching files.	Maintain an indexed design repository and re-inject relevant parts.
Development (Coding)	Provide source files, dependency maps, and coding standards so AI can generate or refactor code.	LLM may “forget” earlier files in large repos.	Use selective file loading; ask AI about only relevant parts at a time.
Testing	Share test cases and coverage reports for automated test generation.	Context overflow with large test suites.	Pass only failing cases or focus area tests into prompt.
Deployment	Review CI/CD config files and release notes.	Non-critical here, but large config sets can still overflow.	Provide modular config snippets.
Maintenance	Bug triage using historical logs and commits.	Logs may exceed token limit.	Summarize logs before feeding to model.

Agents (1/2)

Agents definition (not quite new, indeed quite old):

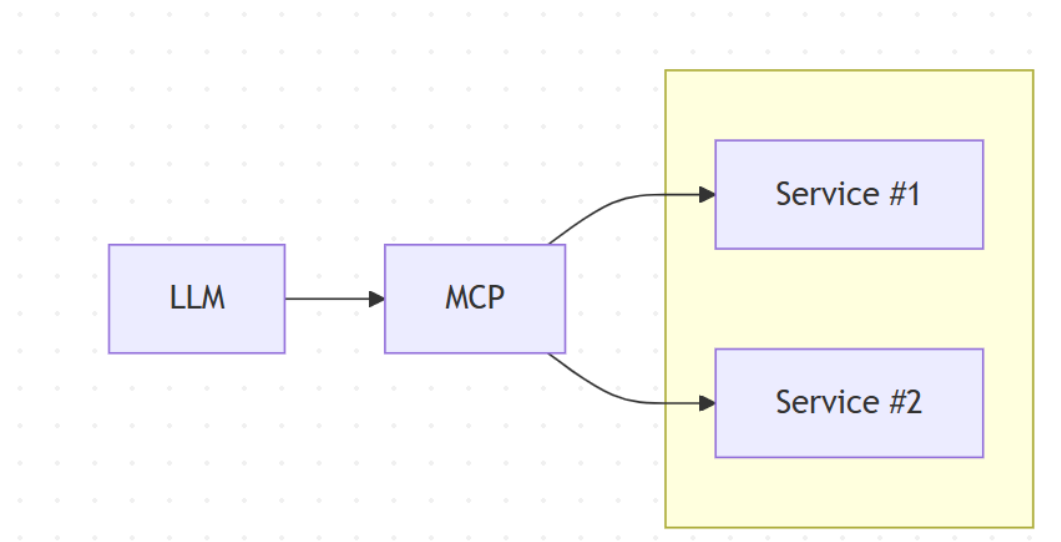
Software system that **ac on behalf of users**;
accomplish **tasks in autonomous way**.

Tools:

- External resources of functions
 - Called upon by agents to accomplish tasks.
-

Agents (2/2)

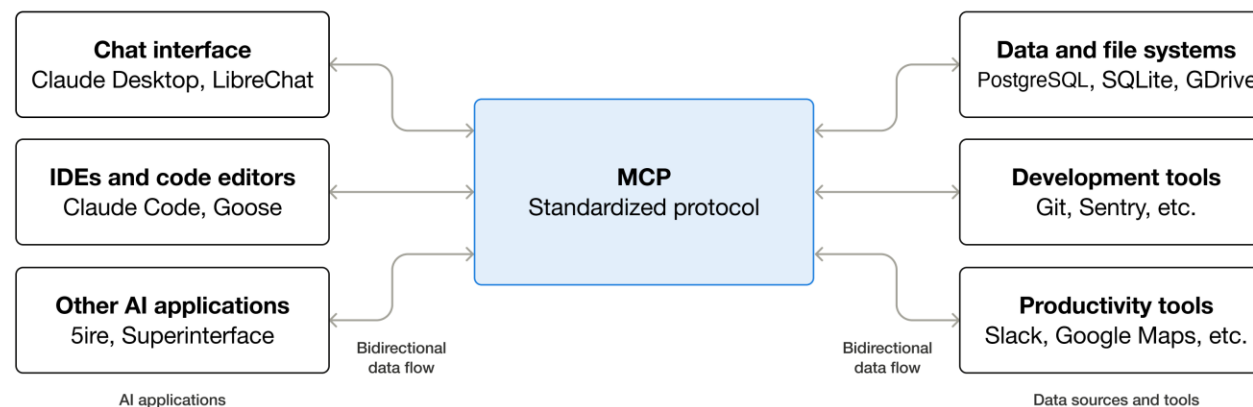
From a simplistic view:



Agents – MCP protocol and ecosystem

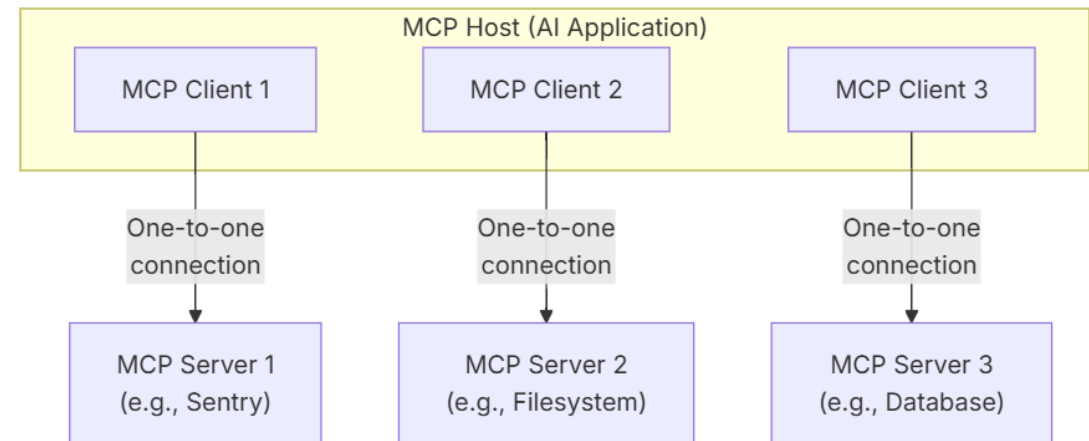
The Agents with MCP are important if they are used in an ecosystem.

For instance, the IDE should be able to connect with Github or JIRA, tools from Observability, Documentation, Communication channels with the team, and other tools.



MCP protocol with Host Architecture

- The MCP Host (your AI application) creates independent one-to-one connections with each MCP Server.
- Each server provides isolated capabilities and each client manages exactly one server connection.
- MCP Servers can be either local or remote, depending on the use-case



Agents (1/2) - Create an Issue



Agents (2/2) - Create an Issue

BMDSoftware/archive-plugins

Create an issue with for Store-n-Forward plugin. The gateway anonymization should be able to trace results and return them to the original source.

 1 reference ▾

A draft issue has been prepared for your request regarding the Store-n-Forward plugin. Please review the details below.

Enable tracing and result return for gateway anonymization in Store-n-Forward plugin

Let me know if you would like to make any changes or provide additional details before finalizing this issue.






How I can help you?

GPT-4.1

Enable tracing and result return for gateway anonymization in Store-n-Forward plugin

Give feedback Create

BMDSoftware/archive-plugins / Feature request

Add a title *

Enable tracing and result return for gateway anonymization in Store-n-Forward plugin

Add a description

Write Preview

H B I [List Icons]

Context and Motivation

The current Store-n-Forward plugin gateway anonymization does not support tracing results or returning them to the original source. There is a need to enhance the plugin so that anonymized results can be tracked and properly routed back to their originator.

Description

As a gateway user,

I want the gateway anonymization process to allow tracing of results, so that returned results can be reliably sent back to the original requester/source.

This will ensure usability and auditability for workflows requiring anonymization but also response tracking.

Acceptance criteria:

- The plugin should support associating anonymized results with their original source.
- The system should be able to route returned results to the correct requester after anonymization.
- Technical details of how tracing is achieved (e.g., via unique identifiers or tags) should be specified.
- The solution should not compromise privacy or security guarantees of anonymization.
- Documentation should be updated to reflect the new tracing and return capabilities.

Paste, drop, or click to add files

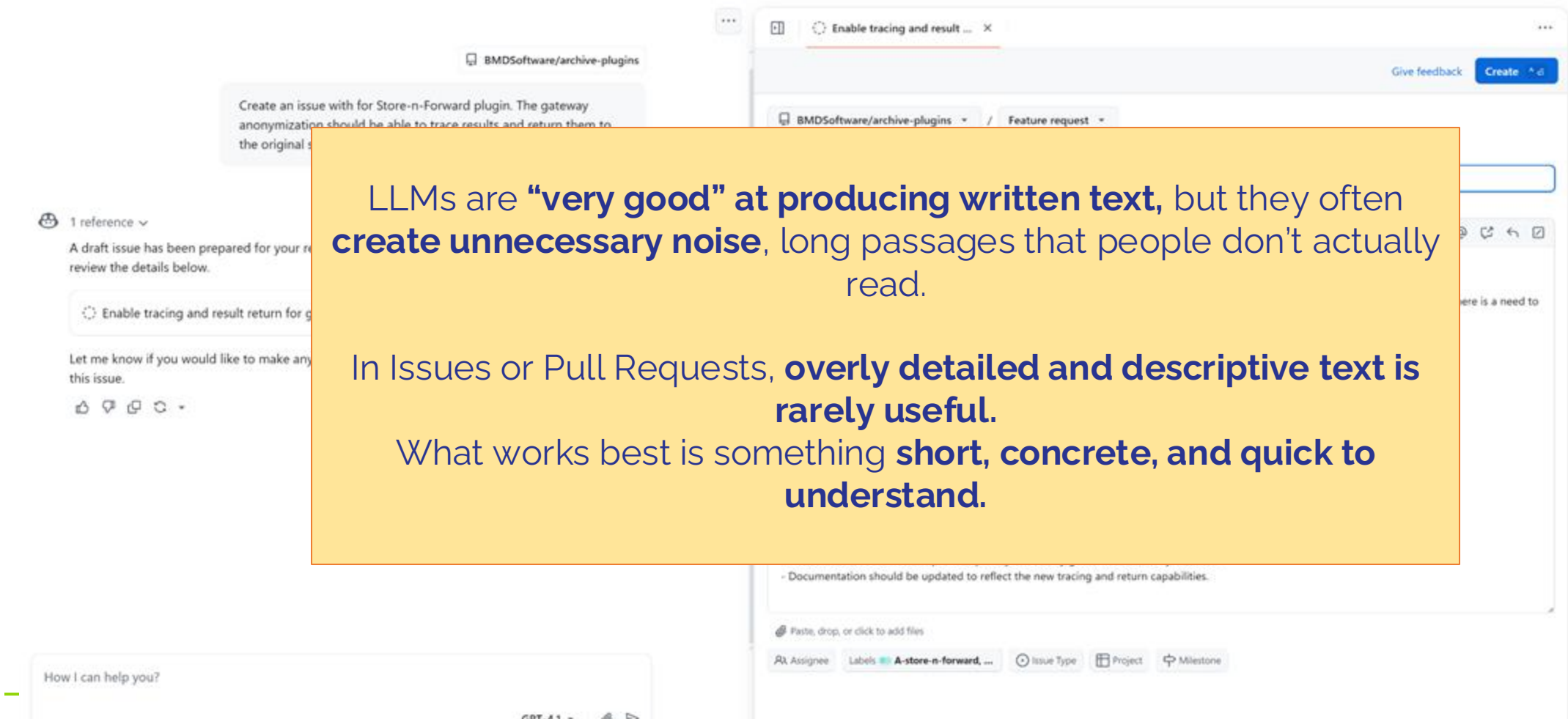
Assignee Labels A-store-n-forward, ... Issue Type Project Milestone

Agents (2/2) - Create an Issue

LLMs are **“very good” at producing written text**, but they often **create unnecessary noise**, long passages that people don't actually read.

In Issues or Pull Requests, **overly detailed and descriptive text is rarely useful.**

What works best is something **short, concrete, and quick to understand.**



Liability

Who is responsible for that code, documentation or issues? The developer.

Risk: AI-generated code **may unintentionally reproduce copyrighted or proprietary code from its training data**, leading to intellectual property disputes.

Introduce Security Issues

Risk: AI may introduce insecure coding patterns or dependencies, increasing the attack surface.

Liability: The organization deploying the code could be held responsible for damages caused by a breach.

Mitigation:

- ▶ Integrate secure code review into the CI/CD pipeline.
- ▶ Follow OWASP AI/LLM Security Guidance (<https://owasp.org/www-project-ai-security-and-privacy-guide/>).
- ▶ Apply static/dynamic analysis tools on AI-generated code.

Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence

Authors:  Jinan Zhang,  Kaifeng Huang,  Yibei Huang,  Bihuan Chen,  Ruoli Wang,  Chong Wang,  Xin Peng [Authors Info & Claims](#)

ACM Transactions on Software Engineering and Methodology, Volume 34, Issue 4 • Article No.: 104, Pages 1 - 28
<https://doi.org/10.1145/3705304>

Published: 28 April 2025 [Publication History](#)



<https://arxiv.org/html/2501.19012v1>

Github Copilot Pro

- Ghost Text completion (*just code*)
- Inline chat (*Ctrl+i*)
- Chat sidebar (*like ChatGPT, but integrated in IDE, fix using copilot, etc*)
- Agents (*create issue, Pull Request Review, etc*)

Universities may include an Education License, so you can use it.

Summary - best practices

- Policy & Training: Learn/Read about on responsible AI use in SDLC.
- Testing & Verification: Never deploy AI outputs without thorough human review.
- Traceability: Document prompts, outputs, and decisions for auditability
- Security & Privacy: Avoid exposing sensitive code/data to external LLMs; Agents with access to sensitive data/resources/tools
- Purpose Use: Use GenAI for assistance (e.g., boilerplate, docs, tests), not for critical logic
- Continuous Improvement: Refine prompts and workflows as the system evolves, but do NOT lose your time refine too much.

Again, this area is undergoing rapid and continuous transformation.

Use self critical thinking.

Bibliography

- All reference materials in the slides

